PROJECT IN ADVANCED ROBOTICS - What We Tried & How We Improved It

Loai Saadi | Michel Ghanadry | Mohammad Mhaniha | GitHub Repo

1) Goal (what we wanted to achieve)

In this project we wanted to estimate the camera motion between two frames. The output we aimed for is a 6-DoF motion: a rotation matrix R (3x3) and a translation vector t (3x1). After that, we extended the same idea to a full video so we can draw the motion path over time.

- Input: two images OR a video (.mp4)
- Output (two images mode): R (rotation) and t (translation direction)
- Output (video mode): a 3D trajectory plot + rotation over time

2) First working solution (two frames)

We started with a simple and standard Visual Odometry pipeline using ORB features. The idea was: find matching points between the two frames, compute the Essential Matrix, and recover the relative pose between the frames.

- Detect keypoints + descriptors (ORB)
- Match descriptors (BFMatcher)
- Compute Essential Matrix E using RANSAC
- Recover pose using recoverPose -> gives R and t

Small code example (core steps):

```
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

matches = bf.match(des1, des2)
pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

E, mask = cv2.findEssentialMat(pts1, pts2, K, method=cv2.RANSAC)
_, R, t, _ = cv2.recoverPose(E, pts1, pts2, K)
```

At this point we already got valid rotation angles and a translation direction. Important note: this t is not in real meters yet - it is only a direction (scale is unknown).

After these naive trials and retries, we noticed that the two-frame result is not always stable (it can change a lot depending on which two frames we choose). So we moved to a video-based approach to get a smoother and more reliable motion path.

3) Moving from 2 frames to a full video

After the two-frame version worked, we moved to videos. We split the .mp4 into frames and then ran the same two-frame estimation many times: frame i -> frame i+1 (or frame i -> frame i+step).

- We extract frames into a folder (frame_000000.jpg, frame_000001.jpg, ...)
- We estimate motion between consecutive frames
- We accumulate translation to build a trajectory over time

We also tested using a larger step (for example step=5) so we get a bigger motion baseline, which usually improves stability.

4) Problems we saw (and why results looked messy)

When we started plotting the 3D path, we noticed the trajectory sometimes looks too noisy or unrealistic even when the real motion is simple (left-right, up-down). This happened for a few reasons:

- Scale ambiguity: VO does not know the real distance without extra info.
- Bad matches: some matched points are wrong, which creates wrong pose estimates.
- Small parallax: if the camera motion is tiny or mostly rotation, the translation is hard to estimate.
- Rolling shutter / stabilization (especially phones): can break the geometric model.

5) What we changed to make the code better

Instead of keeping the first version as-is, we improved it step by step. Each change was small, and we checked if the results became more stable.

5.1 Better matching (less wrong correspondences)

- We replaced simple crossCheck matching with KNN matching + Lowe ratio test.
- We kept only the best matches and ignored weak ones.

5.2 Switching from ORB to SIFT

We tried SIFT because it is usually more stable than ORB in challenging scenes (lighting / blur / bigger changes). We used FLANN matcher for faster and better nearest-neighbor search.

5.3 Stronger RANSAC filtering

- We tuned the RANSAC threshold to reject more outliers.
- We used the inlier mask from findEssentialMat to keep only good points.
- We skipped frames when inliers were too few (instead of adding garbage motion).

5.4 Smoothing the final path

Even with good matching, small noise is expected. So we applied a moving-average smoothing window on the final trajectory (in centimeters).

6) ArUco scale and ground-truth idea

To fight the scale problem, we added ArUco markers. When an ArUco marker is visible in a frame, OpenCV can estimate its pose using the real marker size. That gives us a translation in real units (meters), so we can compare VO vs ArUco and even scale the VO translation.

- We detect the marker and run estimatePoseSingleMarkers.
- We invert marker->camera to get camera position in the marker coordinate system.
- We plot SIFT trajectory (smoothed) together with ArUco trajectory (ground truth).

Sometimes ArUco was empty (no detections). We saw that this happens when the marker is: too small in the image, blurry, overexposed, partially outside the frame, or the dictionary does not match the printed marker.

7) Camera calibration - what we tried and what we used in the end

At first we used a guessed camera matrix K built from the frame size. But we wanted better results, so we tried to calibrate the camera using a ChArUco board video.

- We recorded a calibration video and tried to detect ChArUco corners.
- We repeatedly got: accepted frames = 0 (not enough detected corners).
- So we changed dictionary/parameters and tried again, but still no accepted frames.

Finally, we used an existing file calibration_filtered.npz that already contains:

- K (3x3) - camera intrinsic matrix
- dist (1x5) - distortion coefficients

Example load (used in our scripts):

```
data = np.load('calibration_filtered.npz')
K = data['K']
dist = data['dist']
```

Using the real K and dist made the pose estimation more consistent, because the geometry model matches the real camera.

8) Back from video to 2 frames (final submission script)

After we tried the video pipeline and used it to improve our matching and stability, we came back to the original requirement: estimating the relative pose between two frames only. So we built a clean final script called pose_estimator_2frames.py that takes 2 images, uses SIFT + Essential Matrix (RANSAC), and outputs the Rotation matrix R and Translation vector t.

- Input: --img1 <frame1.jpg> --img2 <frame2.jpg>
- Optional: --calib calibration_filtered.npz (loads K + dist for better accuracy)
- Output: Rotation matrix R (3×3) + Translation vector t (3×1, direction / unscaled)

This is the final function we submit. It uses the same logic we tested in the video version (feature matching → inliers → essential matrix → recoverPose), but now focused on the exact 2-frame task.

9) How to run pose_estimator_2frames.py (command line)

• Example (**with calibration**):

python pose_estimator_2frames.py --img1 "img1.jpg" --img2 "img2.jpg" --calib “calibration.npz”

• Example (**without calibration**, the script estimates K from image size):

python pose_estimator_2frames.py --img1 "img1.jpg" --img2 "img2.jpg"

The script prints R and t directly to the terminal.

10) Summary (what we learned)

We started from a simple two-frame pose estimator, then we pushed it to work on videos and plot motion paths. We improved stability by using better matching (ratio test), switching to SIFT, filtering outliers with RANSAC inliers, and smoothing the final trajectory. We also added ArUco as a real-world scale reference and used a real camera calibration file (K + distortion) to make results more consistent.